

บทที่ 7

หลักการสร้างและออกแบบโปรแกรม

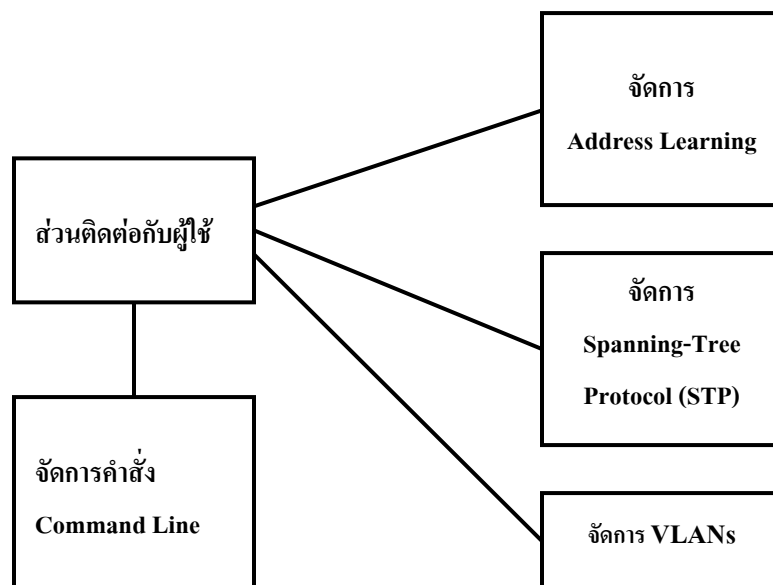
7.1 โครงสร้างของโปรแกรม (Design)

โปรแกรมนี้ ออกแบบโดยใช้หลักการของออบเจกต์โอเรียนเต้ดโปรแกรมมิง (Object Oriented Programming) เพื่อให้โปรแกรมง่ายต่อการเพิ่มเติมและดัดแปลงโปรแกรม โดยแบ่งการทำงานของโปรแกรมออกเป็นส่วนย่อย ๆ เป็นคลาสต่าง ๆ แต่ละคลาสจะมีหน้าที่การทำงานที่แตกต่างกันดังนี้

- *Frame1* เป็นคลาสสำหรับสร้างหน้าต่างอินเทอร์เฟซเพื่อติดต่อกับผู้ใช้ โดยหน้าต่างอินเทอร์เฟซจะประกอบไปด้วยเมนูบาร์ ทูลบาร์ และส่วนที่ใช้แสดงภาพเครือข่ายจำลองในรูปแบบกราฟฟิกส์
- *Interface* เป็นคลาสที่ประกอบไปด้วยข้อมูลของอินเทอร์เฟซแต่ละตัว เช่น ชื่ออินเทอร์เฟซ ประเภทอินเทอร์เฟซ และสถานะของอินเทอร์เฟซ
- *Workstation* เป็นคลาสที่ประกอบไปด้วยข้อมูลต่างๆ ของคอมพิวเตอร์ ได้แก่ ชื่อคอมพิวเตอร์ และค่าแมคแอดเดรส
- *Dialog4* เป็นคลาสสำหรับให้ผู้ใช้เชื่อมต่อสวิตช์แต่ละตัวเข้าด้วยกัน หรือเชื่อมต่อคอมพิวเตอร์เข้ากับสวิตช์
- *AddressLearning* เป็นคลาสสำหรับการเรียนรู้ค่าแมคแอดเดรส
- *Switch* เป็นคลาสที่ประกอบไปด้วยข้อมูลต่างๆ ของสวิตช์ เช่น ชื่อสวิตช์ อินเทอร์เฟซทั้งหมดของสวิตช์ ตารางการเรียนรู้ค่าแมคแอดเดรส โดยคลาสนี้จะมีฟังก์ชันการทำงานสำหรับเรียนรู้ค่าแมคแอดเดรส ฟังก์ชันการทำงานสำหรับรับ-ส่งเฟรมทั้งที่เป็น เฟรมอีเทอร์เน็ตและวีแลนเฟรม และฟังก์ชันการทำงานสำหรับรับ-ส่งเฟรมที่ใช้ในการทำสเปนนิงทรีโปรโตคอล
- *SwitchImg* เป็นคลาสที่เก็บข้อมูลของภาพสวิตช์แต่ละตัว
- *ComImg* เป็นคลาสที่เก็บข้อมูลของภาพคอมพิวเตอร์แต่ละตัว
- *InterfaceImg* เป็นคลาสที่เก็บตำแหน่งของอินเทอร์เฟซแต่ละตัวที่มีการเชื่อมต่อ
- *Vlan* เป็นคลาสที่ใช้ในการเก็บข้อมูลของแต่ละวีแลนว่ามีสมาชิกเป็นสวิตช์ตัวใด อินเทอร์เฟซ (Interface) ไหนบ้าง รวมไปถึงเก็บค่าของเปอร์วีแลนสเปนนิงทรี (Per-VLAN Spanning Tree) ของวีแลนด้วย
- *VlanDatabase* เป็นคลาสที่เก็บข้อมูลของวีแลนแต่ละวีแลน เช่น หมายเลขวีแลน ชื่อวีแลน
- *SpanningTree* เป็นคลาสที่ใช้ในการสร้างสเปนนิงทรีให้กับเครือข่ายโดยควบคุมการทำงานของสวิตช์ทุกตัว
- *Panel2* เป็นคลาสที่ใช้สำหรับสร้างเครือข่าย และแสดงผลในรูปแบบของกราฟฟิกส์ (Graphics)

- *SwitchCMD* เป็นคลาสที่ใช้สำหรับรับคำสั่งจากผู้ใช้มาตรวจสอบว่าเป็นคำสั่งที่ถูกต้องก่อนจะไปเลือกการทำงานของแต่ละคำสั่ง ซึ่งจะมีฟังก์ชันหลักเป็นตัวตรวจสอบว่าเป็นคำสั่งที่ถูกต้อง และส่งไปทำงานตามแต่ละคำสั่ง
- *SwitchConsole* เป็นคลาสสำหรับให้ผู้ใช้ป้อนคำสั่งต่างๆ ที่ใช้ในการตั้งค่าสวิตช์ และแสดงผลออกมาในรูปแบบของ Text Mode
- *Panel1* เป็นคลาสสำหรับแสดงสถานะของสวิตช์แต่ละตัว และสถานะของอินเทอร์เฟซต่างๆ ของสวิตช์

โดยคลาสเหล่านี้สามารถแบ่งตามการจัดการได้ดังนี้

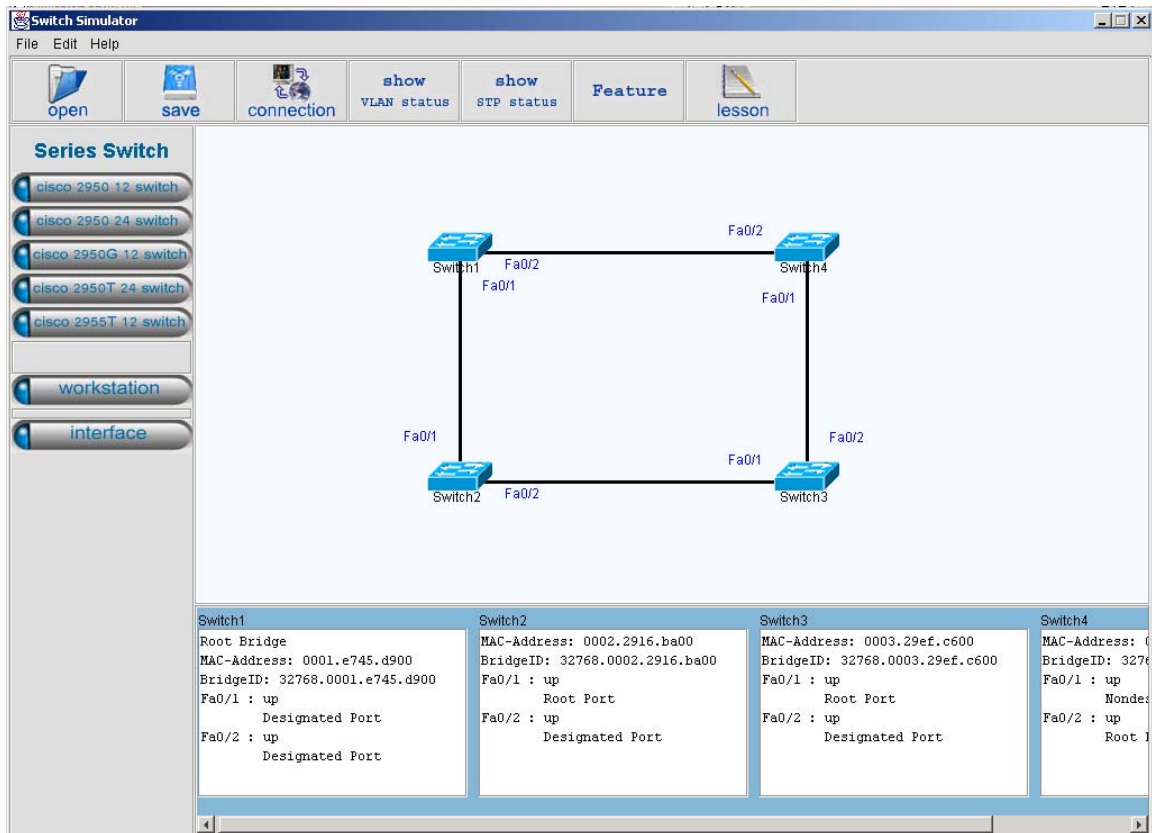


รูปที่ 7-1 การออกแบบโปรแกรม

7.2 การจัดการส่วนติดต่อกับผู้ใช้ (Graphic User Interface : GUI)

แบ่งออกเป็น 3 ส่วนหลักๆ คือ

1. ส่วนหน้าจอหลักของโปรแกรม ประกอบด้วยทูลบาร์ ส่วนของหน้าจอแสดงแผนภาพเครือข่าย และส่วนแสดงสถานะของสวิตช์ และอินเทอร์เฟซของสวิตช์แต่ละตัว การออกแบบส่วนนี้จะแบ่งเป็น 3 คลาส คือ คลาส *Frame1* ซึ่งเป็นหน้าจอหลัก คลาส *Panel2* ซึ่งเป็นหน้าจอแสดงผลภาพรวมของเครือข่าย โดยจะแสดงการเชื่อมต่อของสวิตช์และคอมพิวเตอร์แต่ละตัวตามที่กำหนด นอกจากนี้ผู้ใช้สามารถเพิ่มสวิตช์ คอมพิวเตอร์ และสร้างการเชื่อมต่อระหว่างสวิตช์กับสวิตช์ หรือสวิตช์กับคอมพิวเตอร์ได้ คลาส *Panel1* ซึ่งเป็นส่วนแสดงสถานะของสวิตช์แต่ละตัว เมื่อมีการเพิ่มสวิตช์เข้าไปในระบบก็จะมีการสร้างออบเจกต์ของ *Panel1* ของสวิตช์ตัวนั้นขึ้นมาด้วย แล้วนำมาเรียงกันในหน้าจอหลักของโปรแกรม



รูปที่ 7-2 หน้าจอหลักของโปรแกรม

ฟังก์ชันการทำงานสำหรับส่วนนี้ประกอบไปด้วยคลาส Frame1, Panel1 และ Panel2 มีฟังก์ชันการทำงานดังนี้

คลาส Frame1

- Frame1() เป็นคอนสตรัคเตอร์ของคลาส ในฟังก์ชันนี้จะแสดงหน้าจอโปรแกรมหลักทั้งส่วนของเมนูบาร์ และทูลบาร์
- newSwitch() เป็นฟังก์ชันสำหรับสร้างสวิตช์แต่ละตัว โดยจะมีการตรวจสอบชนิดของสวิตช์ที่เลือกก่อน และตรวจสอบจำนวนสวิตช์ทั้งหมดด้วยว่าสามารถเพิ่มสวิตช์เข้าไปในระบบได้อีก
- ส่วนฟังก์ชันอื่นๆ เป็นฟังก์ชันสำหรับการจัดการการกระทำของผู้ใช้สำหรับแต่ละคอมโพเนนต์ ซึ่งเป็นการจัดการการกระทำที่ผู้ใช้กระทำกับคอมโพเนนต์หนึ่งๆ ในขณะที่คอมโพเนนต์หนึ่งๆ ถูกโฟกัสอยู่ เช่น jButton1_actionPerformed() เป็นการจัดการกับการกระทำของผู้ใช้กับปุ่มเพิ่มสวิตช์

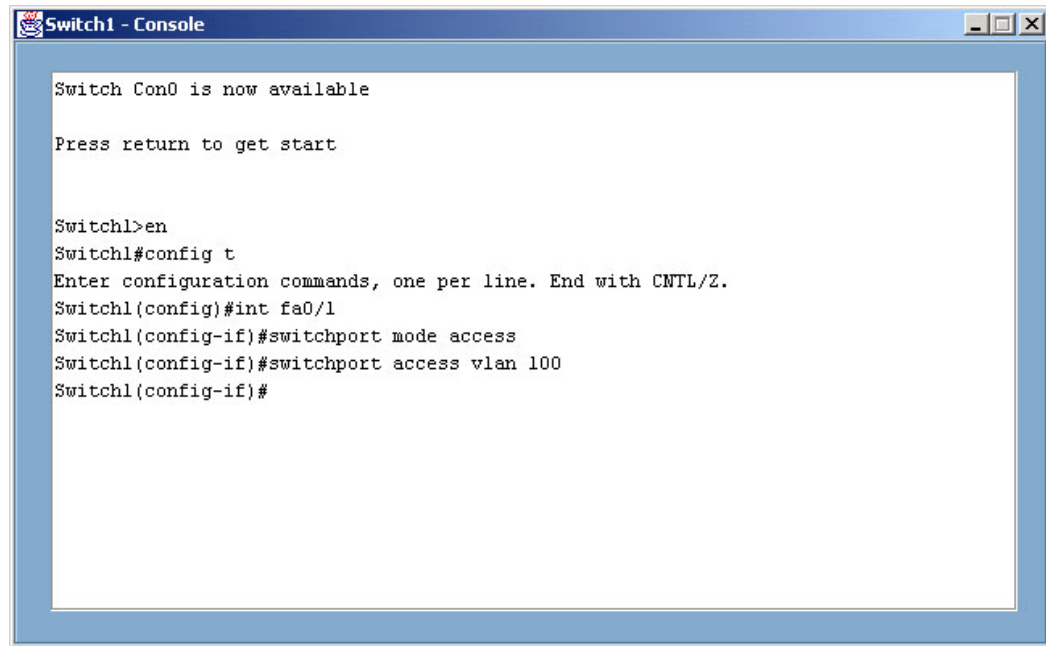
คลาส Panel1

- Panel1() เป็นคอนสตรัคเตอร์ของคลาส
- showDetail() เป็นฟังก์ชันสำหรับแสดงรายละเอียดของสวิทช์แต่ละตัว รวมไปถึงรายละเอียดของอินเทอร์เฟซของสวิทช์นั้นๆ ด้วย

คลาส Panel2

- Panel2() เป็นคอนสตรัคเตอร์ของคลาสสำหรับแสดงผลภาพรวมของเครือข่าย
- delSwitch_actionPerformed() เป็นการจัดการกับการกระทำของผู้ใช้ กับเมนูป๊อปอัพ (Menu popup) สำหรับลบสวิทช์
- delComp_actionPerformed() เป็นการจัดการกับการกระทำของผู้ใช้กับเมนูป๊อปอัพ สำหรับลบคอมพิวเตอร์
- delInt_actionPerformed() เป็นการจัดการกับการกระทำของผู้ใช้กับเมนูป๊อปอัพ สำหรับลบอินเทอร์เฟซ
- console_actionPerformed() เป็นการจัดการกับการกระทำของผู้ใช้กับเมนูป๊อปอัพ สำหรับแสดงคอนโซลของสวิทช์ตัวที่เลือก
- connect_actionPerformed() เป็นการจัดการกับการกระทำของผู้ใช้กับเมนูป๊อปอัพ สำหรับเปลี่ยนอินเทอร์เฟซที่ใช้ในการเชื่อมต่อ
- this_mousePressed() เป็นฟังก์ชันจัดการกับการคลิกเมาส์
- this_mouseReleased() เป็นฟังก์ชันจัดการกับการปล่อยเมาส์
- this_mouseDragged() เป็นฟังก์ชันจัดการกับการคลิกเมาส์ และเลื่อนเมาส์ไปพร้อมกัน

2. หน้าจอสำหรับแสดงคอนโซลของสวิตช์แต่ละตัว เมื่อมีการเพิ่มสวิตช์เข้าไปในระบบก็จะมีการสร้างออบเจกต์ของ SwitchConsole ของสวิตช์ตัวนั้นขึ้นมาด้วย

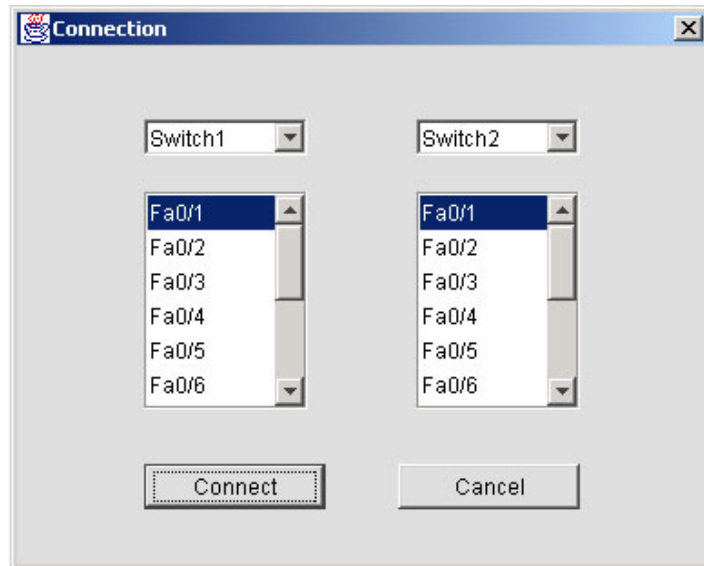


รูปที่ 7-3 หน้าจอสำหรับแสดงคอนโซลของสวิตช์

ฟังก์ชันการทำงานสำหรับส่วนนี้จะอยู่ในคลาส SwitchConsole มีฟังก์ชันการทำงานดังนี้

- SwitchConsole() เป็นคลาสคอนสตรัคเตอร์ของคลาสทำหน้าที่แสดงส่วนที่จะนำหน้าจอไปวางบนแท็บเพน
- ShowPrompt() เป็นฟังก์ชันสำหรับแสดงพร้อมพ์ โดยจะดูจากโหมดการทำงานของสวิตช์แล้วจึงแสดงพร้อมพ์ของโหมดการทำงานนั้นๆ สำหรับฟังก์ชันนี้จะรับอาร์กิวเมนต์เข้าไปเป็นสตริงเพื่อนำไปแสดงผลต่อจากพร้อมพ์
- ส่วนฟังก์ชันอื่นๆ เป็นฟังก์ชันที่จัดการกับการกระทำที่เกี่ยวกับคีย์บอร์ดและเมาส์ เช่น ฟังก์ชัน JTextArea1_keyPressed() จัดการการกระทำที่เกี่ยวกับคีย์บอร์ดในขณะที่ TextArea ถูกโฟกัสอยู่ ฟังก์ชัน JTextArea1_MouseClicked() จัดการการกระทำของการคลิกเมาส์ในขณะที่ TextArea ถูกโฟกัสอยู่ และฟังก์ชัน JTextArea1_mouseReleased() จัดการกับการกระทำของการปล่อยเมาส์ ทำให้ผู้ใช้ไม่สามารถเลือกข้อความได้

3. หน้าจอสำหรับเชื่อมต่อสวิตช์แต่ละตัวเข้าด้วยกัน หรือ เชื่อมต่อสวิตช์กับคอมพิวเตอร์เข้าด้วยกัน โดยผู้ใช้สามารถเลือกสวิตช์ 2 ตัว ที่จะเชื่อมต่อเข้าด้วยกัน หรือเลือกสวิตช์และคอมพิวเตอร์ที่จะเชื่อมต่อเข้าด้วยกัน เมื่อเลือกสวิตช์แล้ว สามารถเลือกพอร์ตของสวิตช์ในการเชื่อมต่อได้



รูปที่ 7-4 หน้าจอสำหรับเชื่อมต่อสวิตช์แต่ละตัวเข้าด้วยกัน หรือเชื่อมต่อคอมพิวเตอร์เข้ากับสวิตช์

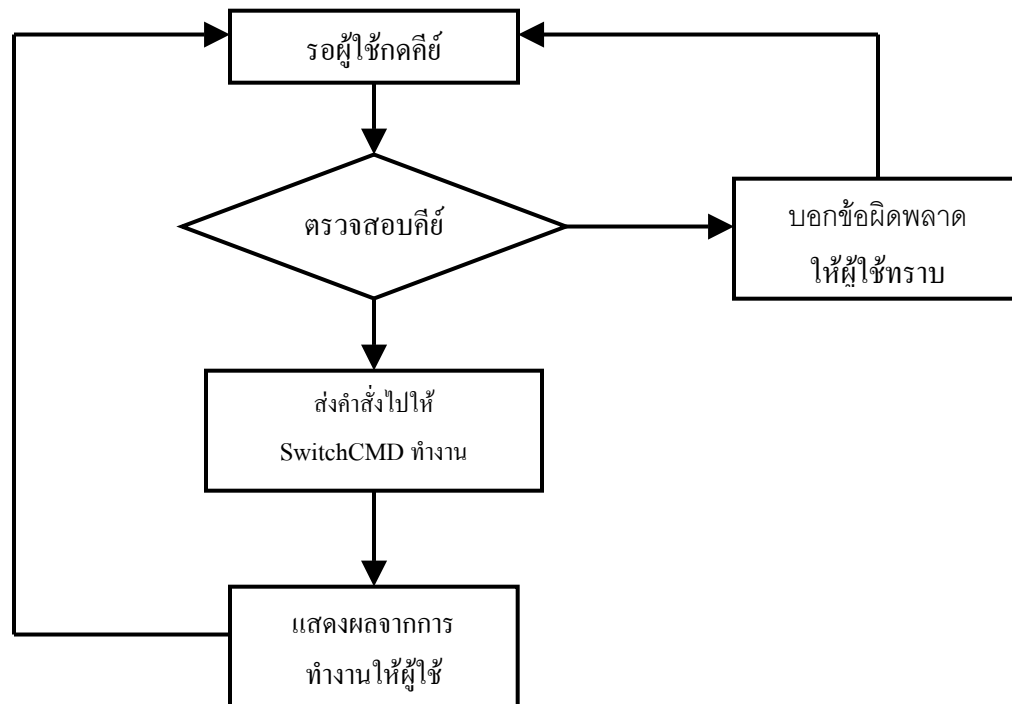
ฟังก์ชันการทำงานสำหรับส่วนนี้จะอยู่ในคลาส Dialog4 มีฟังก์ชันการทำงานดังนี้

- Dialog4() เป็นคอนสตรัคเตอร์ของคลาสสำหรับแสดงหน้าจอนี้
- InitValue() เป็นฟังก์ชันที่ใช้สำหรับแสดงสวิตช์ทั้งหมดให้ผู้ใช้เลือกเพื่อเชื่อมต่อสวิตช์เข้าด้วยกัน
- jButton1_actionPerformed() เป็นฟังก์ชันสำหรับจัดการการกระทำของผู้ใช้กับปุ่ม Connect
- jButton2_actionPerformed() เป็นฟังก์ชันสำหรับจัดการการกระทำของผู้ใช้กับปุ่ม Cancel
- ส่วนฟังก์ชันอื่นๆ เป็นฟังก์ชันสำหรับจัดการการกระทำของผู้ใช้ที่เกี่ยวกับการเลือกสวิตช์แต่ละตัว เช่น jComboBox1_actionPerformed() เป็นการจัดการให้แสดงพอร์ตทั้งหมดที่สามารถใช้ได้ของสวิตช์ตัวแรก que ผู้ใช้เลือก และแสดงสวิตช์ตัวที่เลือกให้ผู้ใช้เลือกสวิตช์ตัวที่สองที่ต้องการเชื่อมต่อเข้ากับสวิตช์ตัวแรก
- ส่วนฟังก์ชันอื่นๆ เป็นฟังก์ชันสำหรับจัดการการกระทำของผู้ใช้ที่เกี่ยวกับการเลือกสวิตช์ เช่น jComboBox1_actionPerformed() เป็นการจัดการให้แสดงพอร์ตทั้งหมดที่สามารถใช้ได้ของสวิตช์ตัวแรก que ผู้ใช้เลือก และแสดงพอร์ตทั้งหมดที่สามารถใช้ได้ของสวิตช์ตัวที่เลือก

7.3 การจัดการส่วนของ Command Line

มีคลาสที่จัดการส่วนนี้ 1 คลาส คือ คลาส SwitchCMD จะคอยรับคำสั่งที่ส่งมาจากคลาส Frame1 ที่จัดการในส่วนที่ติดต่อกับผู้ใช้ เมื่อรับคำสั่งมาแล้วจะนำมาตรวจสอบว่าเป็นคำสั่งที่ถูกต้อง สามารถทำงานได้หรือไม่ ถ้าถูกต้องจะส่งคำสั่งไปทำงาน และรอรับผลการทำงาน เพื่อนำมาแสดงผลให้ผู้ใช้ทราบ

ขั้นตอนการทำงานของโปรแกรม



รูปที่ 7-5 แสดงขั้นตอนการจัดการกับส่วน Command Line

ฟังก์ชันการทำงานสำหรับส่วนจะอยู่ในคลาส SwitchCMD มีฟังก์ชันการทำงานดังนี้

คลาส SwitchCMD

- runCommand() เป็นฟังก์ชันหลักในการทำงานของคลาสนี้ คำที่รับเข้ามาในฟังก์ชันนี้จะมีอยู่ 3 คำคือ สวิตช์ คำสั่งที่ต้องการทำงานกับสวิตช์นั้นๆ และคอนโซลของสวิตช์นั้นๆ โดยเริ่มต้นที่ดูที่โหมดการทำงานของสวิตช์เพื่อเลือกชุดคำสั่งที่สามารถทำงานได้กับโหมดนั้น หลังจากนั้นจะตัดคำมาทีละคำเพื่อเปรียบเทียบกับคำสั่งที่มีทุกคำสั่งในโหมดนั้น ถ้าเจอคำสั่งที่สอดคล้องกันก็จะนับจำนวนคำสั่งนั้นไว้ เมื่อเปรียบเทียบครบทุกคำสั่ง ก็จะนำค่าของจำนวนคำสั่งที่สอดคล้องกันมาตรวจสอบว่ามากกว่า 1 คำสั่งหรือไม่ ถ้ามากกว่าก็จะไม่เลือกคำสั่งใดทำงาน และรายงานผลกลับไปให้กับผู้ ถ้าเท่ากับ 1 ก็จะส่งไปให้ฟังก์ชันของแต่ละคำสั่งทำงาน

- ฟังก์ชันอื่นๆ จะเป็นฟังก์ชันการทำงานของคำสั่งแต่ละคำสั่ง

7.4 การจัดการในส่วนของการรับ-ส่งเฟรม และการเรียนรู้ค่าแมคแอดเดรสของสวิตช์

การจัดการในส่วนของการรับ-ส่งเฟรมจะมีคลาส AddressLearning เป็นคลาสหลักในการจัดการ โดยคลาสนี้มีการสืบทอด (Inherit) มาจากคลาสแม่ที่เป็นเธรด (Thread) เนื่องจากมีการจัดการให้คอมพิวเตอร์มีการส่งเฟรมตลอดเวลา เพื่อให้สวิตช์มีการเรียนรู้ค่าแมคแอดเดรส ดังนั้นการแตกโปรเซสให้โปรแกรมจัดการให้คอมพิวเตอร์ส่งเฟรม และสวิตช์มีการเรียนรู้ค่าแมคแอดเดรสเป็นโปรเซสแบบเธรด ทำให้สิ้นเปลืองเวลาน้อยกว่ารอให้โปรแกรมทำงานเอง

ในการรับและส่งเฟรมของคอมพิวเตอร์ เฟรมที่รับและส่งจะมีลักษณะเป็นเฟรมอีเทอร์เน็ต ส่วนการรับและส่งเฟรมของสวิตช์ เฟรมที่รับและส่งจะมี 2 ประเภท คือ เฟรมอีเทอร์เน็ต และ เฟรมที่เป็นวีแลน แต่เนื่องจากเป็นโปรแกรมที่จำลองเฟรมอีเทอร์เน็ตจึงมีเฉพาะค่าแมคแอดเดรสของคอมพิวเตอร์ต้นทาง (Source Address) และค่าแมคแอดเดรสของคอมพิวเตอร์ปลายทาง (Destination Address) ส่วนเฟรมที่เป็นวีแลนจะเพิ่มข้อมูลในส่วนที่ระบุวีแลนเข้าไปในเฟรมอีเทอร์เน็ต

สำหรับการเรียนรู้ค่าแมคแอดเดรสของสวิตช์ เมื่อสวิตช์ได้รับเฟรมมาแล้ว จะนำค่าแมคแอดเดรสต้นทางของเฟรมนั้นมา แล้วค้นหาค่าแมคแอดเดรสในตารางเรียนรู้ค่าแมคแอดเดรส ซึ่งในตารางจะเก็บค่าแมคแอดเดรส และพอร์ตที่เชื่อมต่อกับแมคแอดเดรสต่างๆ โดยสามารถตรวจสอบได้ดังนี้

1. ถ้าในตารางไม่มีแมคแอดเดรสนี้ โปรแกรมจะเพิ่มแมคแอดเดรส และพอร์ตที่รับเฟรมนี้มา ลงในตาราง
2. ถ้าในตารางมีแมคแอดเดรสนี้ จะตรวจสอบพอร์ตที่รับเฟรมนี้มากับพอร์ตที่ตรวจพบในตารางของแมคแอดเดรสนี้ ถ้าพอร์ตไม่ตรงกัน จะเปลี่ยนพอร์ตในตารางเป็นพอร์ตที่รับเฟรมนี้มา

สำหรับการส่งเฟรมของสวิตช์ มีขั้นตอนการทำงานของโปรแกรดังนี้

1. ถ้าเฟรมที่ได้รับมาเป็นเฟรมอีเทอร์เน็ต โปรแกรมจะตรวจสอบว่าพอร์ตที่ได้รับเข้ามามีค่าวีแลนเป็นอะไร จากนั้นนำค่าแมคแอดเดรสไปค้นหาในตารางเรียนรู้ค่าแมคแอดเดรส
 - ถ้าพบแมคแอดเดรสในตาราง จะตรวจสอบว่าพอร์ตที่ระบุไว้เป็นวีแลนเดียวกันกับพอร์ตที่รับเข้ามา ถ้าเป็นวีแลนเดียวกันสวิตช์จะส่งเฟรมนี้ออกไปทางพอร์ตที่ระบุในตาราง
 - ถ้าไม่พบแมคแอดเดรสในตาราง สวิตช์จะส่งเฟรมนี้ออกไปทุกพอร์ต
2. ถ้าพอร์ตที่ส่งออกไปเป็นแอ็กเซสลิงค์ จะตรวจสอบว่าเป็นวีแลนเดียวกันก่อนจึงส่งออกไป แต่ถ้าเป็นทริงค์ลิงค์ โปรแกรมจะเพิ่มข้อมูลที่ระบุวีแลนเข้าไปในเฟรมนั้น และตรวจสอบว่าวีแลนนี้สามารถส่งผ่านทริงค์ลิงค์นี้ได้ก่อนที่จะส่งออกไป

ฟังก์ชันการทำงานจะอยู่ในคลาส AddressLearning คลาส Switch และคลาส Workstation มีฟังก์ชันการทำงานดังนี้

คลาส AddressLearning

- run() เป็นฟังก์ชันสำหรับให้เทรตทำงาน โดยจะทำงานทุกๆ 1 วินาที เพื่อเรียกให้ฟังก์ชันของคอมพิวเตอร์ส่งเฟรมออกไป

คลาส Switch

- accessMACAddressTable() เป็นฟังก์ชันสำหรับเรียนรู้ค่าแมคแอดเดรส
- sendFrame() ฟังก์ชันนี้จะตรวจสอบว่าเชื่อมต่ออยู่กับสวิตช์ และคอมพิวเตอร์ตัวใดบ้าง และตรวจสอบกับตารางแมคแอดเดรส ตรวจสอบพอร์ตที่ใช้ในการส่งเฟรม แล้วจึงเรียกฟังก์ชัน receiveFrame() ของคลาส Switch เพื่อให้สวิตช์รับเฟรมมาเก็บไว้ และฟังก์ชัน receiveFrame() ของคลาส Workstation เพื่อให้คอมพิวเตอร์รับเฟรม
- receiveFrame() ฟังก์ชันนี้จะรับเฟรมเข้ามา แล้วเรียกฟังก์ชัน accessMACAddressTable() เพื่อเรียนรู้ค่าแมคแอดเดรส

คลาส Workstation

- sendFrame() ฟังก์ชันนี้จะตรวจสอบว่าเชื่อมต่ออยู่กับสวิตช์ตัวใด แล้วจึงเรียกฟังก์ชัน receiveFrame() ของสวิตช์ตัวนั้นเพื่อรับเฟรม
- receiveFrame() เป็นฟังก์ชันสำหรับรับเฟรมจากสวิตช์

7.5 การจัดการในส่วนของสเปนนิงทรีโพรโตคอล

การจัดการในการสร้างสเปนนิงทรีเพื่อทำการกำจัดลูปในระบบเครือข่ายนั้น จะใช้คลาส SpanningTree เป็นคลาสหลักในการจัดการ ควบคุมการทำงานของสวิตช์ทั้งหมด เพื่อทำการสร้างสเปนนิงทรีขึ้นในระบบเครือข่าย โดยคลาส SpanningTree จะทำหน้าที่สั่งการให้สวิตช์ทั้งหมดเริ่มต้นรับ-ส่ง BPDU เพื่อทำการเรียนรู้และสร้างสเปนนิงทรี

ฟังก์ชันการทำงานของสเปนนิงโพรโตคอลจะอยู่ในคลาส SpanningTree และคลาส Switch โดยมีฟังก์ชันในการทำงานดังนี้

คลาส SpanningTree

- enableCommonSpanningTree() เป็น ฟังก์ชัน ที่ใช้ในการ สั่งงาน สวิตช์ ทั้งหมด โดยเรียก doCommonSpanningTree() ของสวิตช์และควบคุมการสร้างคอมมอนสเปนนิงทรี (Common Spanning Tree -- เป็นการกำจัดลูปของทั้งเครือข่าย) จนเสร็จสิ้น
- disableCommonSTP() เป็นฟังก์ชันที่ใช้ในการยกเลิกคอมมอนสเปนนิงทรีที่เคยสร้างไว้ โดยทำการ ตั้งค่าต่าง ๆ ให้กลับไปเป็นเหมือนตอนเริ่มต้น

- enablePerVLANSpanningTree() เป็นฟังก์ชันที่ใช้ในการสั่งงานสวิตช์ทั้งหมดที่เป็นสมาชิกของวีแลน โดยเรียก doPerVLANSpanningTree() ของสวิตช์และควบคุมการสร้างเปอร์วีแลนสเปนนิ่งทรี (Per-VLAN Spanning Tree -- เป็นการกำจัดลูปของวีแลนที่กำหนด) จนเสร็จสิ้น
- disablePerVLANSTP เป็นฟังก์ชันที่ใช้ในการยกเลิกเปอร์วีแลนสเปนนิ่งทรีที่เคยสร้างไว้ โดยทำการตั้งค่าต่าง ๆ ให้กลับไปเป็นเหมือนตอนเริ่มต้น

คลาส Switch

- doCommonSpanningTree() เป็นฟังก์ชันที่ใช้ในส่ง-รับ BPDU's ให้กับสวิตช์ข้างเคียงทั้งหมด โดยเรียกฟังก์ชัน receiveBPDU() เพื่อรับ BPDU's ไปคำนวณ
- receiveBPDU() เป็นฟังก์ชันที่ใช้ในการรับ BPDU มาเพื่อทำการคำนวณค่าต่าง ๆ เพื่อสร้างคอมมอนสเปนนิ่งทรี
- doPerVLANSpanningTree() เป็นฟังก์ชันที่ใช้ในส่ง-รับ BPDU's ให้กับสวิตช์ข้างเคียงที่เป็นสมาชิกของวีแลนเดียวกัน โดยเรียกฟังก์ชัน receiveBPDU() เพื่อรับ BPDU's ไปคำนวณ
- receiveBPDUforVlan() เป็นฟังก์ชันที่ใช้ในการรับ BPDU มาเพื่อทำการคำนวณค่าต่าง ๆ เพื่อสร้างเปอร์วีแลนสเปนนิ่งทรี (Per-VLAN Spanning Tree)